

Tecnologico de Costa Rica
Escuela de Computacion
Principios de Sistemas Operativos

Segundo Proyecto

Libreria e interfaz de linea de comando para indexar archivos
JSON grandes

Profesor: Armando Arce Orozco
Estudiantes: Nicole Parra, 2023223291
Alfredo Mercado, 2023377984

Junio 2026

Índice

1. Introduccion	1
2. Analisis del problema	1
3. Solucion propuesta	1
3.1. Interfaz de la libreria	2
3.2. Estructuras internas	2
3.3. Construccion del indice	2
3.4. Manejo de rutas	3
3.5. Busqueda sobre el indice	3
3.6. Formato de salida	4
3.7. Manejo de errores y memoria	4
3.8. Uso del programa	5
4. Formato del indice JNX	5
5. Ejecucion de pruebas	5
6. Analisis de resultados	6
7. Limitaciones	7
8. Conclusiones	7

1. Introduccion

El proyecto implementa una libreria en C y un programa de linea de comando para indexar archivos JSON. La idea principal es parecida a un archivo ISAM: se mantiene el archivo de datos original y se crea un archivo auxiliar con las rutas de los valores y sus posiciones en bytes. Con esto, una busqueda posterior no necesita parsear todo el JSON de nuevo, sino revisar el indice y extraer el fragmento directamente con `fseek`.

La solucion fue pensada para archivos JSON bien formados. No se uso una libreria externa para parsear el JSON, porque el enunciado pide escribir un parser propio. El parser cubre objetos, arreglos, strings, numeros, booleanos y `null`.

2. Analisis del problema

Cuando un archivo JSON crece, leerlo completo en cada consulta puede ser costoso. El problema se puede separar en dos fases:

- Construir una lista de rutas absolutas para todos los valores del JSON.
- Guardar, para cada ruta, el byte inicial y el byte final del fragmento dentro del archivo original.

Luego, las consultas se resuelven aplicando una expresion regular sobre las rutas del indice. Si una ruta coincide, se usan sus offsets para recuperar el fragmento exacto del JSON original. Esta division permite pagar el costo de parseo una vez durante la construccion del indice y reutilizarlo en varias busquedas.

3. Solucion propuesta

La solucion implementada consiste en construir una libreria en C que pueda ser usada por otros programas y, encima de esa libreria, un ejecutable pequeno en C++ que funciona como interfaz de linea de comando. La idea fue separar la logica importante del proyecto de la parte que solo lee argumentos y muestra resultados. De esta forma, el parser, el indice y la busqueda quedan en C, que es lo solicitado por el enunciado, mientras que el archivo `jqindex.cpp` solo se encarga de usar esa libreria desde terminal.

La implementacion se divide en tres archivos principales:

- `libjsonindex.h`: define la interfaz publica de la libreria.
- `libjsonindex.c`: contiene el parser, la construccion del indice, la busqueda con `regex.h` y la extraccion con `fseek`.
- `jqindex.cpp`: implementa el programa de linea de comando.

3.1. Interfaz de la libreria

El archivo `libjsonindex.h` expone las funciones que necesita el programa principal. Las funciones mas importantes son:

- `jnx_build_index`: recibe la ruta de un archivo JSON y genera el archivo `.jnx`. Si no se recibe una ruta para el indice, se usa el mismo nombre del JSON pero cambiando la extension.
- `jnx_search`: abre el indice, aplica una expresion regular sobre las rutas guardadas y devuelve los fragmentos encontrados.
- `jnx_extract_fragment`: usa `fseek` para colocarse en el byte inicial del fragmento y lee hasta el byte final.
- `jnx_default_index_path`: construye la ruta por defecto del archivo `.jnx`.
- `jnx_last_error`: permite consultar el ultimo error ocurrido dentro de la libreria.

Esta interfaz permite que el programa `jqindex` no tenga que saber como se parsea el JSON ni como se recorren las rutas del indice. Solo llama a la funcion correcta segun el comando recibido.

3.2. Estructuras internas

Dentro de `libjsonindex.c` se usan estructuras simples para almacenar la informacion mientras se construye el indice:

- `JnxEntry`: representa una entrada del indice. Guarda la ruta, el byte inicial y el byte final.
- `JnxEntries`: funciona como un arreglo dinamico de entradas. Crece con `realloc` cuando se necesita mas espacio.
- `Parser`: guarda el texto completo del JSON, la posicion actual del parser y la lista de entradas generadas.

Se decidio usar arreglos dinamicos porque no se conoce de antemano cuantas rutas tendra el archivo JSON. Cada vez que se encuentra un nuevo valor, se llama a una funcion auxiliar que copia la ruta y agrega una entrada al arreglo. Al final de la construccion, esas entradas se escriben en el archivo `.jnx`.

3.3. Construccion del indice

La construccion del indice empieza leyendo el archivo JSON completo en memoria con modo binario. Esto es importante porque los offsets se guardan como posiciones de byte del archivo original. Si el archivo se leyera con alguna conversion de texto, las posiciones podrian no coincidir con lo que luego `fseek` necesita.

Despues de leer el archivo, el parser llama a una funcion general llamada `parse_value`. Esta funcion revisa el caracter actual y decide que tipo de valor debe procesar:

- Si encuentra `{`, llama al parser de objetos.
- Si encuentra `[`, llama al parser de arreglos.
- Si encuentra comillas dobles, procesa un string.
- Si encuentra `t`, `f` o `n`, procesa `true`, `false` o `null`.
- En los demas casos intenta procesar un numero JSON.

Cuando se procesa un objeto, primero se agrega una entrada para el objeto completo. Luego se leen sus llaves una por una. Cada llave se agrega a la ruta actual para formar una nueva ruta hija. Por ejemplo, si el parser esta en `$/usuarios/0` y lee la llave `nombre`, la nueva ruta queda como `/usuarios/0/nombre`. En los arreglos se hace algo parecido, pero el componente de ruta es el numero de posicion: 0, 1, 2, etc.

Para los valores escalares, el parser guarda la posicion antes de leer el valor y la posicion despues de terminarlo. Ese par de offsets permite recuperar despues exactamente el mismo texto del archivo original, sin reconstruirlo ni modificarlo.

3.4. Manejo de rutas

Las rutas se manejan con una sintaxis similar a JSON Pointer. La raiz se representa con `$`, pero las rutas internas se comparan normalmente sin ese simbolo inicial. Para evitar ambigüedades, las llaves que contienen `/` o `~` se escapan:

- `/` se convierte en `~1`.
- `~` se convierte en `~0`.

Esto permite que una llave real llamada `rutas/especiales` no se confunda con dos niveles de ruta diferentes. En las pruebas se incluyo un caso de este tipo para comprobar que la busqueda puede encontrar rutas escapadas.

3.5. Busqueda sobre el indice

La funcion de busqueda no vuelve a parsear el JSON. Lo que hace es abrir el archivo `.jnx` y leerlo linea por linea. Cada linea se separa por tabuladores para obtener:

- la ruta del valor,
- el byte inicial,
- el byte final.

La expresion recibida por el usuario se normaliza antes de compilarse. Si el usuario escribe una expresion que empieza con `$`, se elimina ese simbolo para compararla contra las rutas guardadas en el indice. Tambien se ancla la expresion con inicio y fin, usando una forma equivalente a:

```
^(expresion)$
```

Esto evita coincidencias parciales no deseadas. Por ejemplo, una búsqueda para la ruta de nombres no debería devolver una ruta mas profunda como la de un apellido. En terminos de texto, el caso se puede ver asi:

```
/usuarios/[0-9]+/nombre  
/usuarios/0/nombre/apellido
```

La primera expresion no coincide con la segunda ruta completa, porque la implementacion exige que toda la ruta coincida con la expresion regular.

Cuando una ruta coincide, la búsqueda convierte los offsets de texto a numeros y llama a `jnx_extract_fragment`. Esa funcion abre el JSON original, usa `fseek` para moverse hasta el byte inicial y lee la cantidad de bytes necesaria. Asi se obtiene el fragmento original.

3.6. Formato de salida

La salida se construye segun la cantidad de coincidencias:

- Si no hay coincidencias, se devuelve `[]`.
- Si hay una sola coincidencia, se devuelve ese fragmento directamente.
- Si hay varias coincidencias, se devuelve un arreglo JSON con los fragmentos separados por coma.

Esta decision sigue el formato pedido por el proyecto. Por ejemplo, una búsqueda por nombres devuelve un arreglo:

```
["Ana", "Luis"]
```

En cambio, una búsqueda exacta por la ruta de modo de configuracion devuelve solamente el valor encontrado:

```
"auto"
```

3.7. Manejo de errores y memoria

La libreria usa memoria dinamica para leer el archivo, guardar rutas, almacenar fragmentos y construir la respuesta final. Por eso se agrego la funcion `jnx_free`, que permite liberar memoria devuelta por la libreria desde el programa principal. Tambien se libera la memoria interna cuando termina la construccion del indice o cuando una búsqueda falla.

Los errores se guardan en una variable interna de la libreria. Despues, el programa principal consulta esa variable con la funcion `jnx_last_error`. Esto permite que `jqindex` imprima mensajes como “no se pudo abrir el archivo JSON”, “la expresion regular no es valida” o “se esperaba una coma dentro de un arreglo”. No se implemento un sistema de errores demasiado complejo porque el objetivo del proyecto es la indexacion, pero si se considero importante que los fallos no terminaran silenciosamente.

3.8. Uso del programa

```
make
./jqindex build datos.json
./jqindex search datos.json "$/usuarios/[0-9]+/nombre"
```

Tambien se puede indicar manualmente el archivo de indice:

```
./jqindex build datos.json datos.jnx
./jqindex search datos.json "$/config/opciones/modo" datos.jnx
```

4. Formato del indice JNX

El indice se guarda como texto plano con extension `.jnx`. Cada linea tiene tres columnas separadas por tabulador:

```
ruta<TAB>inicio<TAB>fin
```

La ruta usa un formato parecido a JSON Pointer. La raiz se guarda como `$`. En los hijos se usa `/` para separar componentes. Si una llave contiene `/`, se guarda como `~1`; si contiene `~`, se guarda como `~0`. El campo `fin` es exclusivo, por lo que la longitud del fragmento es:

```
longitud = fin - inicio
```

Ejemplo de las primeras entradas producidas por una prueba:

```
$ 0 388
/usuarios 16 221
/usuarios/0 22 86
/usuarios/0/id 29 30
/usuarios/0/nombre 42 47
/usuarios/0/activo 59 63
```

5. Ejecucion de pruebas

Se hicieron pruebas de caja negra sobre el programa de linea de comando. El JSON de prueba incluyo arreglos, objetos anidados, strings, booleanos, `null`, numeros con exponente y llaves que necesitan escape en la ruta. Tambien se probaron errores esperados.

Prueba	Expresion o accion	Resultado esperado
Nombres por regex	<code>\$/usuarios/[0-9]+/nombre</code>	<code>["Ana", "Luis", "Marta"]</code>
Activos en cualquier nivel	<code>\$.*/activo</code>	<code>[true, false, true]</code>
Busqueda exacta	<code>\$/config/opciones/modo</code>	<code>"auto"</code>
Valor null	<code>\$/usuarios/1/nota</code>	<code>null</code>
Numero con exponente	<code>\$/usuarios/2/nota</code>	<code>-12.5e2</code>
Ruta escapada	<code>\$/rutas~1especiales/tilde~0key</code>	<code>"ok"</code>
Sin coincidencias	<code>\$/no/existe</code>	<code>[]</code>
Indice por defecto	<code>build sin nombre de indice</code>	<code>Se genera .jnx</code>
Regex invalida	<code>\$/usuarios/</code>	<code>Error controlado</code>
JSON invalido	<code>Archivo mal formado</code>	<code>Error controlado</code>
Indice inexistente	<code>Buscar con no_existe.jnx</code>	<code>Error controlado</code>

Cuadro 1: Resumen de pruebas funcionales y negativas.

En la ejecucion local pasaron 11 de 11 casos. El indice del JSON complejo genero 27 entradas. Para una entrega en Linux, los comandos base son:

```
make
./jqindex build archivo.json
./jqindex search archivo.json "$/usuarios/[0-9]+/nombre"
```

6. Analisis de resultados

Los resultados muestran que la busqueda recupera fragmentos exactos del archivo original. Por ejemplo, al buscar nombres no se devuelve texto procesado por una estructura auxiliar, sino los strings originales con sus comillas JSON. Lo mismo pasa con booleanos, numeros y null.

La grafica siguiente resume la profundidad de las rutas generadas en la prueba. Esto ayuda a ver que el indice no solo guarda hojas, sino tambien objetos y arreglos intermedios.

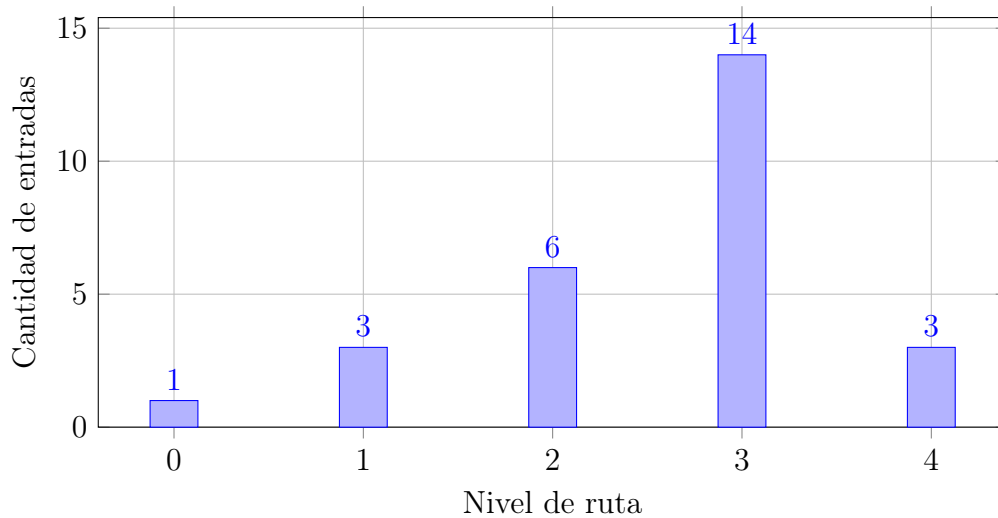


Figura 1: Distribucion de las 27 entradas del indice por nivel de ruta.

La mayor cantidad de entradas quedo en el nivel 3, porque el archivo de prueba tiene varios objetos dentro del arreglo `usuarios`. Este comportamiento es esperable: mientras mas anidado es el JSON, mas rutas absolutas se generan.

7. Limitaciones

El parser asume que el JSON esta bien formado, tal como lo permite el enunciado. No es un parser JSON de produccion. Por ejemplo, no interpreta completamente los caracteres Unicode escapados; para construir rutas reemplaza esos escapes por un caracter simple. Aun asi, para el objetivo del proyecto, el parser reconoce los casos generales necesarios y conserva los offsets de los fragmentos originales.

Otra limitacion practica es que las rutas muy largas podrian exceder el buffer de lectura usado al procesar el archivo `.jnx`. En archivos normales esto no deberia afectar, pero es una condicion que se podria mejorar con lectura dinamica de lineas.

8. Conclusiones

El proyecto cumple con la idea principal del enunciado: crear un indice `.jnx` para un archivo JSON y usarlo para recuperar fragmentos sin parsear el archivo completo en cada busqueda. La libreria en C concentra la logica importante y el programa `jqindex` permite usarla desde la linea de comando.

La prueba mas importante fue comprobar que los offsets guardados realmente sirven para reconstruir valores JSON correctos. Los casos ejecutados cubrieron busquedas con una coincidencia, multiples coincidencias, ausencia de resultados y errores controlados. Con esto se valida que el indice funciona como una estructura auxiliar simple, legible y suficiente para el alcance del proyecto.